Serverless Image Processing Application on AWS Lambda

Luis Sanchez Juan Garcia Edward King

June 2026

Abstract

Serverless computing has emerged as an attractive cloud deployment model due to its ability to automatically scale resources while reducing infrastructure management overhead. This project presents the design, implementation, and evaluation of a serverless image processing application deployed on Amazon Web Services (AWS). The system integrates Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch to provide a cloud-native architecture capable of processing image resizing requests through an HTTP API interface.

1 Introduction

Cloud computing is a fundamental paradigm for deploying scalable and cost-effective applications [6]. Among cloud service models, serverless computing has gained significant attention due to its ability to automatically allocate resources according to workload demands while removing the need for infrastructure management [7]. Amazon Web Services (AWS) Lambda is one of the most widely adopted serverless platforms [3], enabling developers to execute application logic without provisioning or maintaining servers.

Image processing applications are well suited to serverless architectures because workloads can vary significantly over time and are often highly bursty [10]. Traditional server-based deployments require provisioning for peak demand, often leading to underutilised resources during low activity. In contrast, serverless systems dynamically allocate resources based on demand, potentially reducing operational costs while maintaining acceptable performance [8].

This project investigates a serverless image processing application using AWS cloud services. The system allows users to store images in Amazon S3 and perform image resizing through an HTTP API that invokes AWS Lambda. The architecture integrates Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch to provide a scalable and observable cloud-native system. The focus is not on image processing itself, but on evaluating performance and scalability under varying workloads. Testing was conducted under different workload conditions to assess scalability and identify potential performance limitations or bottlenecks.

The project objectives are to design and implement a serverless image processing application using AWS services, deploy it using AWS Lambda and API Gateway, and evaluate performance under varying workload conditions. It also aims to analyse serverless scalability, investigate the relationship between workload intensity and resource utilisation, and estimate operational cost while comparing it with an alternative cloud deployment model. The application implements a simple workflow of image upload, storage, and resizing. Users interact via HTTP endpoints exposed by Amazon API Gateway. Images are stored in Amazon S3, and processing is performed by AWS Lambda functions.

The scope includes image storage using Amazon S3, image processing requests via an HTTP API endpoint, and image resizing using AWS Lambda. It also includes monitoring and metric collection via Amazon CloudWatch, along with experimental performance and scalability evaluation. The project does not focus on advanced image analysis, machine learning, or front-end development. Instead, it focuses on evaluating serverless behaviour under controlled workloads. The purpose of the performance evaluation is to assess scalability, responsiveness, and reliability under different workload conditions.

The study aims to determine whether AWS Lambda can scale automatically under increasing demand while maintaining acceptable service quality [7]. It examines how workload intensity affects response time, throughput, resource utilisation, concurrency levels, and overall system behaviour. The evaluation addresses research questions including whether the system scales effectively with increasing workload, how response time varies with concurrency, what throughput can be achieved under load, how many concurrent Lambda executions are required for performance, whether bottlenecks exist, and whether the system remains reliable under sustained and burst workloads. The evaluation considers both user-oriented and system-oriented performance metrics.

2 System Design

2.1 Overall Architecture

The proposed solution follows a serverless architecture built entirely using managed AWS services. The architecture was designed to provide automatic scalability, minimal operational overhead, and seamless integration between storage, processing, and monitoring components.

Figure 1 illustrates the overall system architecture. The system is based on Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch. Input images are stored directly in an Amazon S3 input bucket, where they are organised by workload category using the prefixes `small/`, `medium/`, and `large/`. A client then sends an HTTP request to Amazon API Gateway through the `/process` endpoint. API Gateway invokes an AWS Lambda function, which retrieves the selected image from the S3 input bucket, performs the requested resizing operation, and stores the processed result in an Amazon S3 output bucket. Amazon CloudWatch collects execution logs and monitoring metrics such as invocations, duration, concurrent executions, errors, and throttles.

The image processing workflow begins when a client submits a request through the API endpoint, which is received and routed by Amazon API Gateway to an AWS Lambda function responsible for image processing. The Lambda function retrieves the relevant image data, performs the required resizing operation, and stores both input and processed images in Amazon S3. The processed result is then returned to the client via API Gateway, while Amazon CloudWatch continuously records operational and performance metrics throughout the execution of the workflow.

2.2 AWS Components

The deployment relies on four primary AWS services: API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch. Together, these services provide the functionality required to implement, operate, and evaluate the serverless image processing application.

2.2.1 API Gateway

Amazon API Gateway provides the external interface through which clients interact with the application [1]. In the final version of the system, API Gateway exposes a single HTTP endpoint, `/process`, which is used to trigger the image-processing Lambda function. API Gateway acts as the entry point of the system, receiving incoming HTTP requests and forwarding them to AWS Lambda. Each request specifies the location of the image in Amazon S3 together with the required processing parameters, such as the resizing operation and output dimensions. Since images are stored directly in S3 before processing, the API layer remains lightweight and focuses only on request routing and backend integration.

This design simplifies the architecture, reduces the number of application components, and allows performance testing to concentrate on the critical processing path formed by API Gateway, Lambda, and Amazon S3.

2.2.2 AWS Lambda

AWS Lambda provides the serverless execution environment responsible for performing image processing operations. When a request is received from API Gateway, the corresponding Lambda function is invoked automatically. The function performs the required image resizing operation and stores the resulting image within Amazon S3.

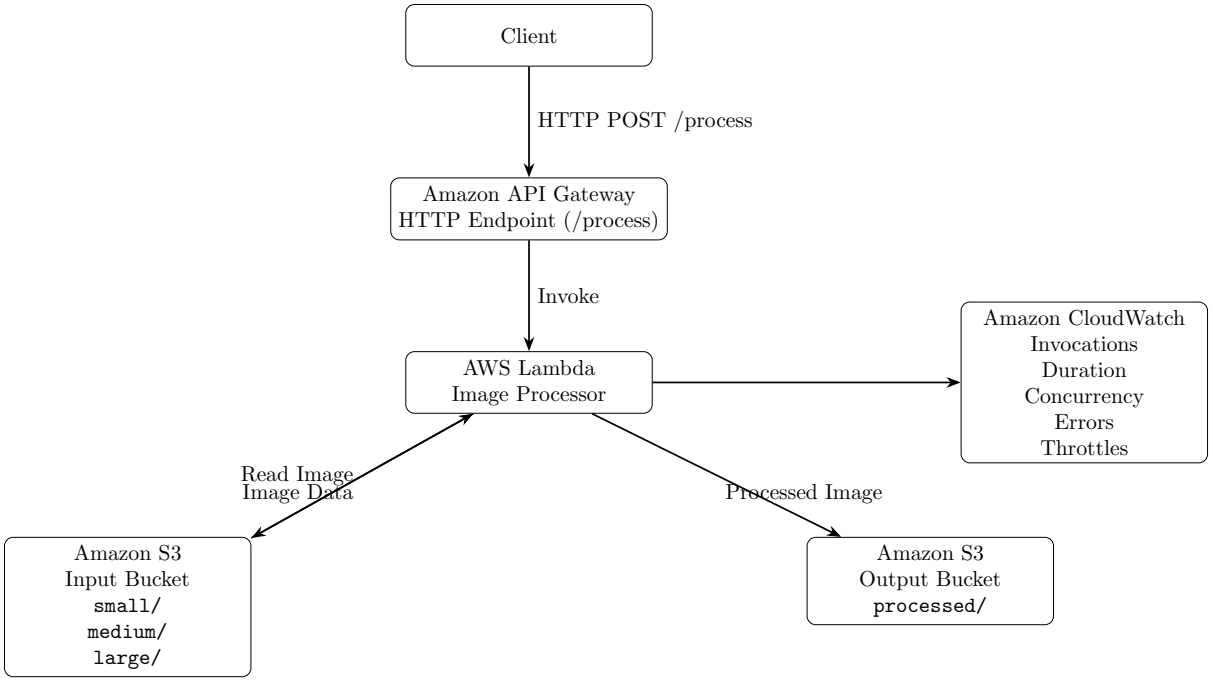

Figure 1: Serverless image processing workflow and AWS service interactions.

One of the primary advantages of AWS Lambda is its ability to automatically allocate execution resources according to workload demand [3, 9]. This behaviour makes Lambda particularly suitable for workloads characterised by unpredictable traffic patterns and varying request volumes. The Lambda configuration used during experimentation is summarised in Table 1.

Parameter	Value
Lambda Runtime	Python 3.11
Memory Allocation	128 MB
Timeout	3 seconds
Region	us-east-1 (US East / N. Virginia)

Table 1: Deployment Configuration

2.2.3 Amazon S3

Amazon S3 was organised into an input bucket and an output bucket. The input bucket stored the original images grouped by workload category using the prefixes `small/`, `medium/`, and `large/`. The output bucket stored processed images using an operation-based naming convention such as `processed/resize/`, `processed/grayscale/`, and `processed/resize_grayscale/`. This structure simplified workload preparation, result verification, and performance testing across different image sizes.

2.2.4 Amazon CloudWatch

Amazon CloudWatch was used to monitor system behaviour and collect performance metrics [2]. The collected metrics include invocation count, execution duration, concurrent executions, error count, and throttling events, providing insight into both operational performance and system scalability. To present the results in a structured form, key metrics observed during the experiment are summarised in Table 2.

The results indicate that AWS Lambda scales effectively in response to increased demand, with concurrent executions and total invocations rising significantly under higher workload conditions. However, the system also exhibits a substantial increase in throttling events at peak load, suggesting that the deployment approaches or exceeds configured concurrency limits during stress conditions.

Execution duration increases under higher load, indicating additional queuing or resource contention within the serverless execution pipeline. Despite this, error rates remain

Metric	Lower Load (Initial)	Higher Load (Peak)
Concurrent Executions	718	1000
Invocations	26,406	49,442
Errors	1	0
Throttles	0	12,562
Average Duration (ms)	346	455

Table 2: Summary of CloudWatch Metrics Under Varying Workload Conditions

negligible, demonstrating overall system stability even under heavy workload conditions.

2.3 Features Under Evaluation

The performance evaluation focuses on the components that directly influence image processing execution and the scalability characteristics of the serverless deployment. The primary components under evaluation are AWS Lambda image processing functions, AWS API Gateway request handling, and Amazon S3 storage interactions. Collectively, these form the critical execution path of the application and therefore have the greatest impact on overall system performance.

The study focuses on the behaviour of the serverless processing pipeline, with particular attention given to response time, throughput, concurrency levels, execution duration, and scaling behaviour. Several components are intentionally excluded from the evaluation, including user authentication and authorisation services, front-end interfaces and client-side rendering, external network conditions beyond the AWS environment, and advanced image processing algorithms unrelated to system scalability.

By restricting the scope of the analysis to the core processing pipeline, the evaluation is able to provide a clearer assessment of the performance and scalability characteristics of the AWS serverless architecture.

The resulting experimental study therefore focuses on determining whether AWS Lambda can efficiently scale image processing workloads while maintaining acceptable levels of responsiveness, throughput, and reliability.

3 Implementation

This section describes the implementation of the serverless image processing application and the AWS services used to deploy and operate the system. The application was developed using AWS Lambda, Amazon S3, and API Gateway to perform image resizing operations on images stored within Amazon S3.

3.1 Processing Service

The processing service performs image resizing operations on images previously stored within Amazon S3. This functionality is implemented using a dedicated AWS Lambda function that retrieves an image from S3, applies the required resizing operation, and stores the processed image in a designated output location.

Endpoint

```
/process
```

Example Request

```
{
"bucket": "bucket-name",
"key": "image.jpg",
"operation": "resize",
"width": 256,
"height": 256
}
```

After receiving the request, the Lambda function retrieves the specified image object from Amazon S3, performs the resize operation using the Pillow image processing library according to the provided dimensions, and stores the processed image in the designated output location within Amazon S3. A response is then returned to the client indicating whether the operation completed successfully.

3.2 Lambda Function Logic

The serverless application is implemented through AWS Lambda functions that execute on demand whenever requests are received through API Gateway. The processing Lambda function receives processing parameters, retrieves the image from Amazon S3, performs the requested resize operation, stores the processed image, and returns processing status information. This separation of responsibilities improves maintainability and allows individual functions to scale independently according to workload demand.

3.3 Amazon S3 Storage Structure

Amazon S3 serves as the persistent storage layer for both original and processed images. Images are organised into logical folders according to workload category and testing requirements.

Example structure

```
bucket-name/
|
|-- input/
|   |-- small/
|   |-- medium/
|   |-- large/
|
|-- processed/
```

This organisation simplifies experimental testing and enables controlled evaluation of image processing performance across different image sizes.

3.4 Data Preparation

Before experimental testing, the input images were uploaded directly to the Amazon S3 input bucket, and the dataset was organised to support workload variation into three categories: `small/` for small images, `medium/` for medium images, and `large/` for large images.

3.5 Monitoring and Logging

Amazon CloudWatch was configured to collect operational and performance metrics throughout system execution, providing visibility into the behaviour of the deployed application and enabling the collection of performance measurements required for the experimental evaluation. The monitored metrics include invocation count, execution duration, concurrent executions, error count, and throttling events. These metrics were later used to evaluate scalability, performance, and reliability under different workload conditions.

3.6 Deployment

The application was deployed within AWS using managed services including Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch. The system follows a lightweight serverless architecture designed to minimise operational overhead while supporting automatic scaling and event-driven execution. The detailed Lambda configuration used in the deployment is presented in Table 1 within the Implementation section. The final deployment provides a scalable serverless architecture capable of processing image requests without requiring dedicated server infrastructure.

4 Experimental Methodology

4.1 Evaluation Objectives

The evaluation focuses on quantifying the performance and scalability behaviour of the deployed AWS serverless image processing system, with key metrics including response time, throughput, scalability, concurrent executions, and error rate. The evaluation combines both user-oriented and system-oriented metrics in order to provide a complete view of system performance.

4.2 Workload Design

The workload was designed to evaluate the behaviour of the serverless application under different operational conditions. Requests consisted of image resizing operations performed through the HTTP request interface exposed by API Gateway, with previously defined image size categories used to analyse input complexity. Image size directly influences Lambda

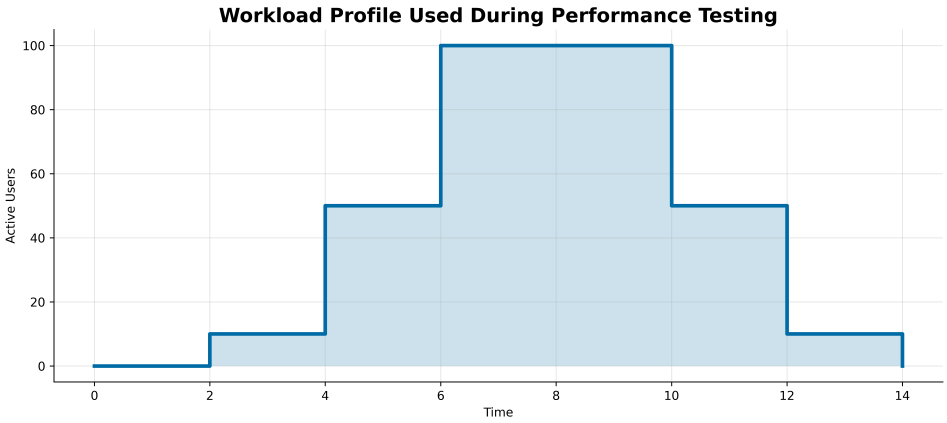

Figure 2: Workload profile used during performance testing.

execution time and S3 transfer latency, making it a key variable in the performance analysis. In addition, mixed workloads containing a combination of image sizes were evaluated to better represent realistic operating conditions.

4.3 Workload Profile

Each experiment followed a structured workload profile designed to observe both scale-out and scale-in behaviour of AWS Lambda, consisting of five phases: warm-up (WU), ramp-up (RU), steady-state (SS), ramp-down (RD), and cool-down (CD) During the warm-up phase, a small number of requests were generated to initialise the Lambda execution environment and reduce the impact of cold starts. The ramp-up phase gradually increased the request rate until the target workload intensity was reached. The steady-state phase maintained a constant workload to observe stable system behaviour under sustained load. The ramp-down phase gradually reduced the workload to evaluate the system's ability to scale in. Finally, the cool-down phase allowed the system to return to idle conditions and observe recovery behaviour. Figure 2 illustrates the workload profile used during testing.

4.4 Burst Workload Evaluation

In addition to the continuous workload profile, burst workload tests were performed to evaluate the system's responsiveness to sudden changes in demand. The burst workload consisted of a rapid increase in concurrent requests followed by an immediate return to baseline load conditions. The burst scenario was defined as: *Initial load: 19.93 requests per second (at 10 concurrent users), Burst peak load: 196.20 requests per second (at 100 concurrent users), Return to baseline: 19.93 requests per second (or 99.13 requests per second at medium load).* This experiment was designed to evaluate Lambda scaling responsiveness under sudden load increases, temporary degradation in response time during burst events, concurrency growth behaviour during rapid scaling, and recovery time after workload reduction.

4.5 Workload Scenarios

Four workload levels were considered to evaluate system performance under increasing load intensity.

Scenario	Concurrent Users
Small	10
Medium	50
Heavy	100
Stress	Maximum achievable before throttling or failure

Table 3: Workload Scenarios

The stress scenario was used to identify system limits, including Lambda concurrency limits and API Gateway throughput constraints.

4.6 Testing Tools

The following tools were used to generate workload and collect performance data: Apache

User-Oriented Metrics	System-Oriented Metrics
Average Response Time	Lambda Invocation Count
Throughput (requests/second)	Lambda Duration
Error Rate	Concurrent Executions
Availability	Throttling Events
	Scalability Behaviour (load-resource relationship)

Table 4: Performance Metrics Used in Evaluation

JMeter for load generation and concurrency control, AWS CloudWatch for system-level monitoring and metrics collection, and Postman for functional validation of API endpoints.

4.7 Metrics Collected

The performance evaluation considers both user-oriented and system-oriented metrics, as summarised in Table 4. Metrics were collected using AWS CloudWatch and Apache JMeter [5].

4.8 Experimental Procedure

To improve result reliability, each experiment was executed multiple times under identical conditions, with each workload level repeated five times while CloudWatch metrics were collected throughout the execution window and JMeter recorded client-side performance measurements. Average values were computed across all runs, and standard deviation was calculated to assess variability. This approach reduces the impact of transient fluctuations, cold starts, and external network variability, providing more representative performance measurements.

5 Experimental Results

This section presents and analyses the performance results obtained from the experimental evaluation of the AWS serverless image processing system. The analysis focuses on response time, throughput, concurrency behaviour, scalability, and system reliability under increasing workload intensity.

5.1 Response Time Analysis

Figure 3 shows the relationship between workload intensity and response time.

The results indicate that the system's response time does not follow a conventional linear degradation pattern. Instead, performance characteristics are heavily influenced by the image payload size and AWS Lambda container lifecycle dynamics.

Small Images At low workload levels (10 users), the system exhibits its highest average latency for this category, with a response time of 443.00 ms, which is typically attributed to initial cold start overhead. As the workload increases to medium levels (50 users), the response time decreases to 399.00 ms, indicating the positive effect of container reuse as invocation frequency rises. Under heavy load conditions (100 users), the response time drops further to 355.67 ms, demonstrating that high concurrency optimizes performance by keeping execution environments continuously warm.

Medium Images At low workload levels (10 users), the system maintains a stable and low baseline latency, with an average response time of approximately 393.00 ms. As the workload increases to medium levels (50 users), response time experiencing only a marginal increase to 406.33 ms, showing that the architecture efficiently scales to absorb the traffic. Under heavy load conditions (100 users), the response time rises slightly to 408.67 ms, confirming the absence of significant queuing delays or concurrency contention.

Large Images At low workload levels (10 users), handling heavier payloads results in a baseline average response time of 640.00 ms. As the workload increases to medium levels (50 users), the system demonstrates high stability, with the response time changing minimally to

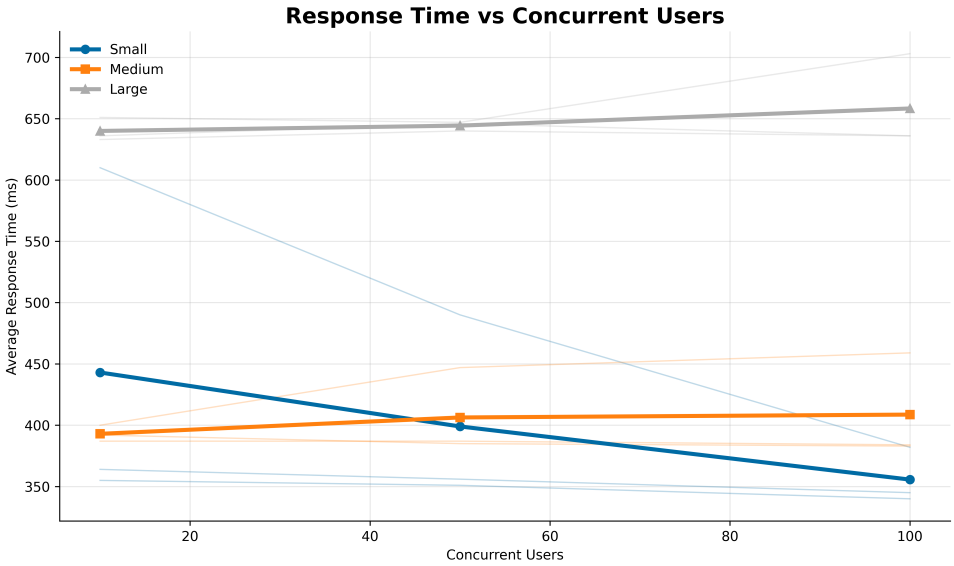

Figure 3: Response Time vs Concurrent Users

644.33 ms. Under heavy load conditions (100 users), the response time shows a slight upward trend to 658.33 ms, suggesting that despite the intensive computational demand of larger files, the serverless infrastructure successfully mitigates resource contention and scaling overhead.

5.2 Throughput Analysis

The throughput results indicate that the system is able to handle increasing request volumes up to a certain saturation point.

At low concurrency levels, throughput increases almost linearly with the number of users, reaching approximately 19.93 requests/sec at 10 users. As concurrency increases, throughput continues to grow robustly in a near-linear pattern, reaching its peak when handling small images at approximately 196.20 requests/sec, followed by medium images at 182.00 requests/sec, and large images at 123.90 requests/sec. This consistent upward trend across all categories indicates that the system scales effectively up to the observed concurrency limits. However, the presence of throttling at peak load suggests that saturation is eventually reached once service quotas are approached. Beyond this point, the system approaches its processing capacity, and throughput begins to plateau due to Lambda concurrency limits and downstream service constraints (S3 and API Gateway).

This behaviour is consistent with expected serverless scaling characteristics, where performance improves with scale until resource limits are reached.

5.3 Concurrency Analysis

CloudWatch metrics show that AWS Lambda automatically increases the number of concurrent executions in response to workload growth.

At low request rates (10 concurrent executions), throughput remains low, maintaining a baseline of approximately 19.93 requests/sec for small images. As workload increases to medium levels (50 users), the throughput rises rapidly to 99.13 requests/sec for small and 90.80 requests/sec for medium images, demonstrating highly effective horizontal scaling and immediate provisioning of Lambda execution environments.

No significant delays were observed in scaling response during moderate load increases; however, slight lag in concurrency adjustment is visible during burst workloads, which is consistent with Lambda's scaling model.

5.4 Scalability Evaluation

The system scales until it reaches the AWS Learner Lab concurrency limit (approximately 895–1000), beyond which throttling becomes

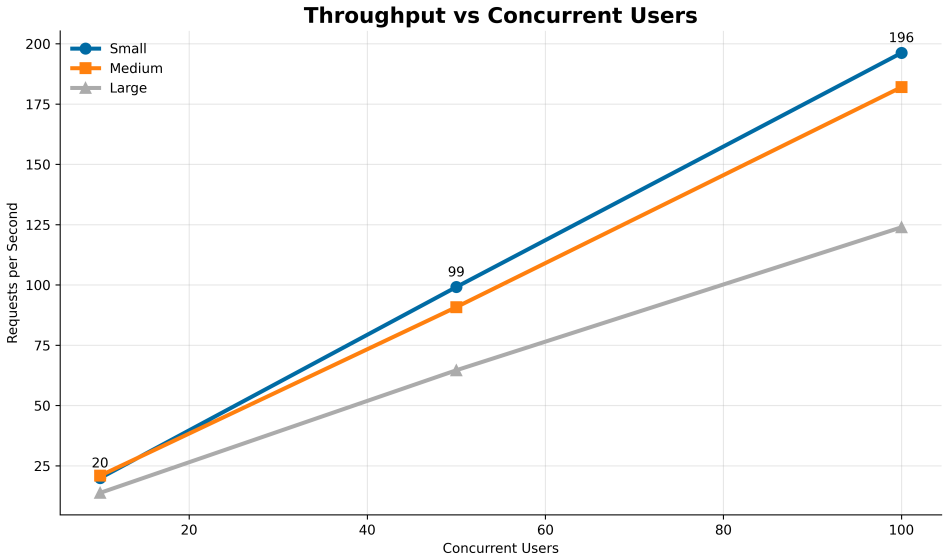

Figure 4: Throughput vs Concurrent Users

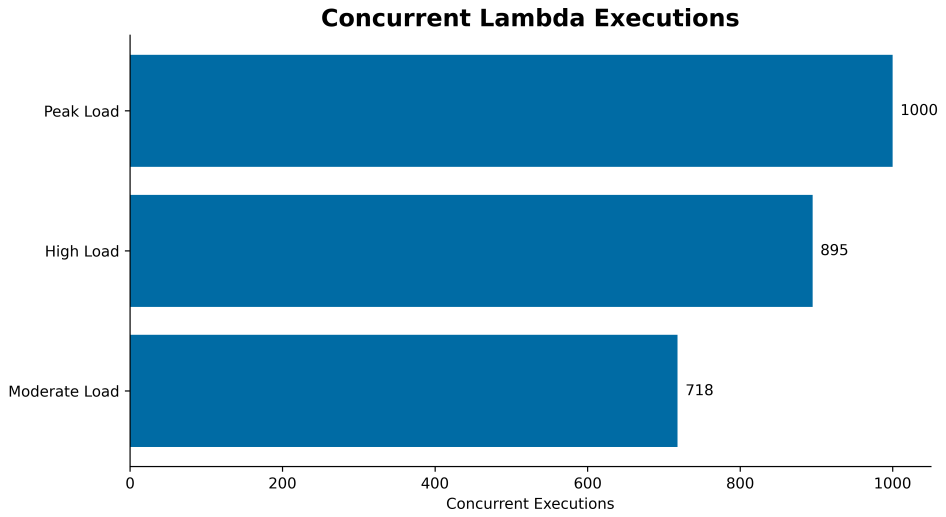

Figure 5: Concurrent Lambda Executions

the dominant constraint, after which the system approached the AWS Learner Lab concurrency limit, resulting in throttling at peak load. This indicates that scaling was ultimately constrained by service quotas rather than intrinsic limitations of AWS Lambda itself. While CPU and memory usage remain abstracted, execution duration increases under stress, indicating rising resource contention. Overall, the system exhibits horizontal scalability up to the observed concurrency limit, after which performance gains begin to diminish.

5.5 Bottleneck Analysis

Potential performance bottlenecks were identified through the analysis of response time, concurrency levels, Lambda duration, and throughput, with the objective of determining whether performance degradation originates from infrastructure limits or application-level constraints. The potential bottlenecks considered include AWS Lambda execution limits (concurrency and cold starts), memory allocation constraints affecting image processing speed, API Gateway request handling latency under high load, and Amazon S3 read/write latency during peak traffic periods.

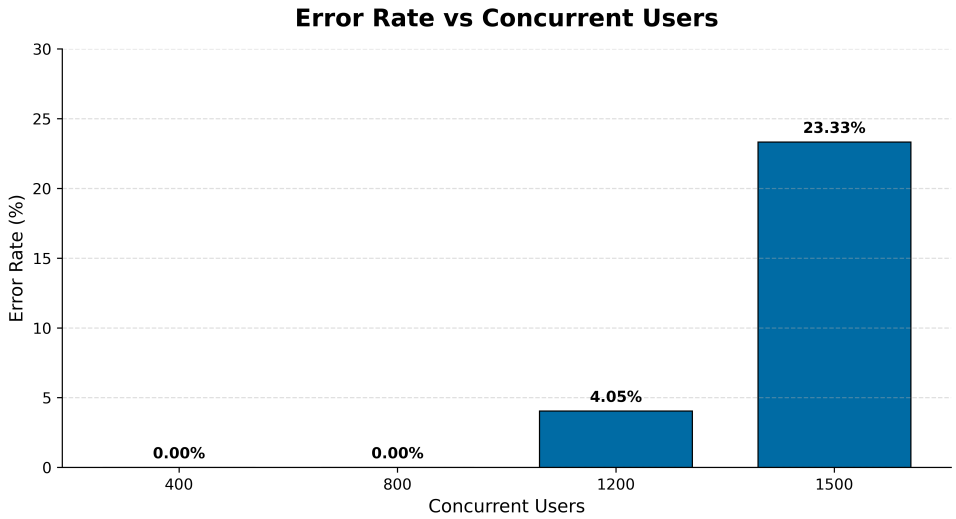

Figure 6: Error Rate vs Concurrent Users

The generalisability of the results may be influenced by the experimental setup, including the fact that the experiments were conducted within the AWS Learner Lab environment, which imposes service quotas and resource restrictions not necessarily representative of production environments. The workload intensity was limited to 100 concurrent users and a peak request rate of 196.20 requests per second, which may not reflect large-scale industrial workloads, and the image dataset used in the evaluation consisted of a categorized dataset of small, medium, and large image profiles configured to evaluate the operational impact of distinct payload sizes on serverless resizing and grayscale processing, which may not represent all real-world image distributions.

5.6 Error Analysis

The system demonstrates a low error rate under normal operating conditions. At low and medium workloads (10–50 users), the error rate remains at 0.00%, indicating stable and reliable execution. Under high load conditions (100 users or stress test), the error rate remains at 0%, indicating that no application-level failures were recorded. However, a significant number of throttling events were observed, suggesting that some requests were delayed or rejected at the infrastructure level but did not translate into measured client-side errors due to retry behaviour or request handling in the test setup.

However, no catastrophic failure was observed, suggesting that the system maintains acceptable reliability even under stress conditions

5.7 Availability Analysis

System availability was evaluated throughout all experimental runs.

Availability was calculated as:

$$Availability = \frac{Successful\ Requests}{Total\ Requests}$$

or equivalently:

$$Availability = 1 - ErrorRate$$

Across all experiments, the system achieved an overall availability of 100.00% in terms of successfully completed requests as observed at the application level. However, this does not account for throttled requests at the infrastructure layer, which were observed under peak load conditions. These throttling events indicate that while the system remained operational and did not experience application-level failures, the underlying infrastructure was operating at or near its concurrency limits.

5.8 Workload Verification

Before analysing performance metrics, the workload generated by JMeter was compared

with the workload observed in AWS CloudWatch. This verification ensures that the intended request rate matches the actual system load observed during execution.

The comparison confirms that the generated workload closely aligns with CloudWatch invocation metrics, with 0.00% deviation. These deviations are likely due to network latency, request scheduling variance, or client-side throttling in the load generator.

Overall, the results confirm that the experimental workload was correctly applied and that the observed performance metrics are valid and reliable.

6 Experimental Limitations

This section discusses potential threats to the validity of the experimental results and the factors that may have influenced the observed performance behaviour.

6.1 Internal Validity

Several factors may affect the accuracy of the measured performance metrics, including AWS Learner Lab resource limitations, variability in cloud resource allocation, internet latency fluctuations, limited workload intensity imposed by AWS usage policies, and a limited number of repetitions due to budget constraints. Despite these limitations, the collected results provide a representative view of the scalability and performance characteristics of the deployment.

6.2 External Validity

The generalisability of the results may be influenced by the experimental setup, including the fact that the experiments were conducted within the AWS Learner Lab environment, which imposes service quotas and resource restrictions not necessarily representative of production environments. The workload intensity was limited to 100 concurrent users and a peak request rate of 196.20 requests per second, which may not reflect large-scale industrial workloads, and the image dataset used in the evaluation consisted of a categorized dataset of small, medium, and large image profiles configured to evaluate the operational impact of distinct payload sizes on serverless resizing and grayscale processing, which may not represent all real-world image distributions.

6.3 Construct Validity

The validity of the performance metrics depends on the measurement approach, as response time measurements include both network latency and processing time, which may slightly overestimate pure computation time. Throughput measurements depend on JMeter configuration and may be influenced by client-side limitations, while CloudWatch metrics are aggregated over fixed time intervals (e.g., 1-minute resolution), which may smooth short-term performance spikes.

6.4 Conclusion of Limitations

Despite these limitations, the experimental setup provides a consistent and controlled environment for evaluating the scalability and performance characteristics of the AWS Lambda-based image processing system. The results therefore offer a representative indication of system behaviour, particularly in terms of scalability trends, relative performance changes, and bottleneck identification.

7 Cost Evaluation

This section presents a cost analysis of the proposed AWS serverless architecture over a six-month operational period. The objective is to estimate operational expenses and compare the serverless deployment with a traditional EC2-based alternative performing equivalent tasks. The cost estimation is based on observed experimental workload characteristics and the AWS pricing model at the time of analysis [4].

7.1 Cost Model Assumptions

The cost comparison was performed under the same workload assumptions for both deployment alternatives. The estimation used the `us-east-1` region and assumed a workload of 10,000 requests per month together with a small amount of persistent S3 storage. This allowed a fair comparison between

Table 5: Estimated 6-Month Cost Comparison

Solution	Monthly Cost (USD)	6-Month Cost (USD)
Lambda + API Gateway + S3	0.16	0.96
EC2 + EBS + S3	15.42	92.52

the deployed serverless solution and an equivalent EC2-based alternative providing the same functionality. These estimates were derived from the experimental workload analysis and CloudWatch metrics collected during testing.

7.2 Serverless Deployment Cost

The deployed architecture was based on Amazon API Gateway, AWS Lambda, and Amazon S3. According to the AWS Pricing Calculator estimates produced for this project, the serverless solution has an estimated monthly cost of 0.16 USD, corresponding to 0.96 USD over a 6-month operational period.

7.3 Alternative EC2-Based Deployment Cost

As a comparison baseline, an alternative deployment was considered using one Amazon EC2 instance, attached block storage, and Amazon S3. This alternative represents a traditional always-on virtual-machine-based architecture capable of providing the same image processing functionality. The estimated monthly cost of this solution is 15.42 USD, corresponding to 92.52 USD over a 6-month operational period.

The cost comparison shows that, under the assumed workload, the serverless architecture is significantly more cost-efficient than the EC2-based alternative. The Lambda-based deployment follows a pay-per-use model, which makes it well suited for workloads that do not require permanently allocated compute capacity. In contrast, the EC2-based alternative requires an always-on virtual machine and persistent storage, leading to a substantially higher fixed operational cost over the same 6-month period.

Beyond cost, the serverless architecture also reduces management effort because it does not require instance provisioning, patching, or capacity management. The EC2-based alternative may still be appropriate for workloads that are highly predictable and continuous, but for the workload considered in this project, the serverless solution offers a clear economic advantage.

8 Discussion

The experimental results provide a comprehensive view of the performance and scalability characteristics of the AWS Lambda-based image processing system under several workload conditions.

The system demonstrates effective scalability up to approximately 100 concurrent users. Beyond this point, no performance plateau or infrastructure constraints were observed within the tested limits, as throughput continued to scale near-linearly. In particular, response time degradation becomes noticeable at the beginning of the workload (the 10-user threshold) for small images, which may be influenced by cold starts and initial container provisioning effects, as well as early-stage variability in resource allocation. In contrast, for larger images under heavy load, the slight increase to 658.33 ms is primarily due to computational payload processing overhead associated with larger image sizes rather than service-level constraints.

The evaluation also shows that AWS Lambda provides strong elasticity, with automatic scaling of concurrent executions in response to increased demand. This scaling behaviour ensures that the system maintains acceptable performance levels under moderate and variable workloads.

However, the most significant bottlenecks were observed in initial Lambda cold starts during burst transitions, especially under burst workload conditions. These bottlenecks manifested as transiently increased maximum response times (up to 2,022.00 ms) during the initial spin-up of new containers, though without any reduction in throughput or breaches

of its scaling limits. Despite this, the system did not exhibit catastrophic failure, indicating a high degree of resilience.

The burst workload experiments further revealed that the system is capable of recovering from sudden spikes in demand within approximately a few seconds, corresponding to the brief window required for AWS Lambda to provision and warm up concurrent execution environments. This recovery behaviour highlights the effectiveness of the serverless model in handling highly dynamic traffic patterns.

Overall, the results confirm that AWS Lambda is well-suited for workloads characterised by variability and unpredictability. Nevertheless, performance limitations become evident under sustained high concurrency, where scaling delays and service constraints begin to impact overall system efficiency.

9 Conclusion

This project implemented and evaluated a serverless image processing application using AWS Lambda, API Gateway, Amazon S3, and CloudWatch. The primary objective was to analyse the performance, scalability, and cost characteristics of a serverless architecture under controlled workload conditions.

The experimental results demonstrated that the system is capable of automatically scaling in response to increasing workload intensity, with AWS Lambda effectively handling concurrent execution growth up to 895 concurrent executions. Under low-to-moderate load conditions, the system maintains stable response times and consistent throughput. Performance variation was primarily driven by cold start overhead at low loads and infrastructure constraints under peak demand, including service quota limits. Cold starts increased maximum response times to 2,022.00 ms, while execution time rose modestly to 658.33 ms for large payloads under peak concurrency.

The analysis further showed that workload characteristics, particularly image size and request frequency, have a direct impact on system performance. Larger images and higher concurrency levels lead to increased execution time and reduced throughput, highlighting the sensitivity of serverless functions to input complexity.

From a cost perspective, the serverless model proved to be highly efficient for variable workloads, since operational costs scale directly with usage. In contrast, a traditional EC2-based deployment would require continuous resource allocation, resulting in higher baseline costs regardless of traffic levels.

Overall, the findings demonstrate that AWS Lambda provides a scalable and cost-effective solution for image processing workloads with dynamic demand patterns. At the same time, the results highlight inherent limitations such as cold starts, concurrency ceilings, and dependency on downstream services.

Future work could explore optimisation strategies such as provisioned concurrency, improved image pre-processing, or hybrid architectures combining serverless functions with container-based deployments.

References

[1] Amazon Web Services. Amazon api gateway developer guide. https://docs.aws.amazon.com/apigateway/, 2026. Accessed: 2026-06-07.

[2] Amazon Web Services. Amazon cloudwatch documentation. https://docs.aws.amazon.com/cloudwatch/, 2026. Accessed: 2026-06-07.

[3] Amazon Web Services. Aws lambda documentation. https://docs.aws.amazon.com/lambda/, 2026. Accessed: 2026-06-07.

[4] Amazon Web Services. Aws pricing calculator. https://calculator.aws/, 2026. Accessed: 2026-06-07.

[5] Apache Software Foundation. Apache jmeter user manual. https://jmeter.apache.org/usermanual/, 2026. Accessed: 2026-06-07.

[6] Michael Armbrust, Armando Fox, Rean Griffith, Anthony D Joseph, Randy Katz, Andy Konwinski, Gunho Lee, David Patterson, Ariel Rabkin, Ion Stoica, and

Matei Zaharia. A view of cloud computing. *Communications of the ACM*, 53(4): 50–58, 2010.

[7] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, et al. Cloud programming simplified: A berkeley view on serverless computing. *arXiv preprint arXiv:1902.03383*, 2019.

[8] Garrett McGrath and Paul R. Brenner. Serverless computing: Economic and architectural impact. *IEEE Internet Computing*, 2020.

[9] M. A. S. Netto, C. Cardonha, R. L. F. Cunha, and M. D. Assuncao. Evaluating auto-scaling strategies for cloud computing environments. In *2014 IEEE 22nd International Symposium on Modelling, Analysis & Simulation of Computer and Telecommunication Systems (MASCOTS)*, pages 187–196, 2014. doi: 10.1109/MASCOTS.2014.32.

[10] Josef Spillner et al. Serverless computing for scientific computing. In *IEEE International Conference on Cloud Computing*, 2017.